Informe de Desarrollo — Sistema de Gestión de Inventario Institucional

> **Proyecto de Grado** · Abril 2026
> Plataforma móvil y web para el control y trazabilidad de activos fijos institucionales.

Tabla de Contenidos

1. Etapa de Análisis
2. Etapa de Diseño
3. Etapa de Codificación
4. Etapa de Pruebas e Implementación

---

1. Etapa de Análisis

1.1 Contexto y Necesidad

Las instituciones educativas y organizaciones similares administran grandes volúmenes de activos fijos (equipos de cómputo, mobiliario, maquinaria, etc.) sin contar con un mecanismo digital centralizado que permita registrar, consultar, auditar y controlar el ciclo de vida de cada bien. Los procesos se realizaban en hojas de cálculo o registros físicos, lo que generaba:

- Duplicidad de códigos de inventario.
- Imposibilidad de trazabilidad histórica de cambios.
- Procesos de toma física de inventario lentos y propensos a errores.
- Falta de control en solicitudes de baja y mantenimiento.

1.2 Técnicas de Recolección de Información

Técnica	Descripción
Entrevistas	Sesiones con el área Administrativa y Financiera para identificar flujos de aprobación y roles de usuario.
Observación directa	Revisión del proceso de toma física de inventario con hojas de papel y códigos manuscritos.
Revisión documental	Análisis de formatos Excel y normativas internas de control de activos.
Benchmarking	Estudio de sistemas similares (ERPs) para identificar funcionalidades clave aplicables a la escala institucional.

1.3 Requerimientos Funcionales

ID	Requerimiento
RF01	El sistema debe permitir registrar activos fijos con código único, categoría, ubicación, responsable, valor y foto.
RF02	El sistema debe mantener un historial inmutable de cambios por cada activo (quién, cuándo, qué).
RF03	El sistema debe autenticar usuarios con control de intentos fallidos y bloqueo temporal de cuenta.
RF04	El sistema debe soportar roles diferenciados: Auxiliar, Administrador, Responsable de Área, DAF, Auditor, Soporte TI.
RF05	El sistema debe permitir realizar sesiones de toma física de inventario, registrando el resultado por activo (encontrado, reubicado, no encontrado, etc.).
RF06	El sistema debe soportar escaneo de códigos QR / barras para identificar activos en campo.
RF07	El sistema debe gestionar solicitudes de mantenimiento preventivo y correctivo.
RF08	El sistema debe gestionar solicitudes de baja con flujo de aprobación por dependencia y DAF.
RF09	El sistema debe generar reportes exportables en PDF y Excel filtrados por ubicación, categoría, estado y responsable.
RF10	El sistema debe funcionar en Android, iOS y Web sin cambios en el servidor.

1.4 Requerimientos No Funcionales

ID	Requerimiento
RNF01	**Seguridad:** contraseñas almacenadas con bcrypt (pgcrypto). Comunicación sobre HTTPS en producción.
RNF02	**Disponibilidad:** backend desplegado en contenedor Docker con health check automático.
RNF03	**Usabilidad:** interfaz Material Design 3 con soporte para escaneo de cámara.
RNF04	**Escalabilidad:** arquitectura por capas que permite migrar a HTTPS, agregar módulos o reemplazar la BD sin tocar la UI.
RNF05	**Persistencia local:** en ausencia de red, la app mantiene datos locales con SharedPreferences (modo demo).

1.5 Diagrama de Casos de Uso (principal)

```
┌──────────────────────────────────────────────────────────────────┐
│                      SISTEMA DE INVENTARIO                       │
│                                                                   |
│  [Iniciar sesión]  ←── Auxiliar / Administrador / Auditor /      │
│  [Ver dashboard]         DAF / Responsable de Área               │
│  [Registrar activo] ◄── Auxiliar · Administrador                 │
│  [Escanear QR/código]◄── Auxiliar · Administrador                │
│  [Toma de inventario]◄── Auxiliar · Administrador                │
│  [Solicitar mantenimiento]◄── Cualquier rol con acceso           │
```

```
│  [Solicitar baja] ◄──── Auxiliar                                  │
│  [Aprobar baja]   ◄──── DAF                                       │
│  [Generar reporte]◄──── Auditor · DAF · Administrador             │
│  [Gestionar usuarios]◄── Administrador                            │
└──────────────────────────────────────────────────────────────────┘
```

1.6 Historias de Usuario (selección)

HU	Como...	Quiero...	Para...
HU-01	Auxiliar de Inventario	Registrar un activo escaneando su código de barras	Acelerar el ingreso y evitar errores tipográficos
HU-02	Administrador	Bloquear un usuario tras N intentos fallidos	Proteger el sistema de accesos no autorizados
HU-03	Auditor	Consultar el historial completo de un activo	Verificar trazabilidad y detectar inconsistencias
HU-04	DAF	Aprobar o rechazar solicitudes de baja	Mantener control financiero sobre los bienes
HU-05	Auxiliar	Conducir una sesión de toma física con escaneo	Realizar el conteo sin papel y en tiempo real

---

2. Etapa de Diseño

2.1 Arquitectura del Sistema

Se seleccionó una **Arquitectura por Capas (N-Tier Layered Architecture)** con cuatro niveles claramente delimitados:

```
┌──────────────────────────────────────────────────────────┐
│  CAPA DE PRESENTACIÓN  ·  Flutter (Dart)                 │
│  Widgets · AppState (ChangeNotifier) · HTTP Client       │
├──────────────────────────────────────────────────────────┤
│  CAPA DE API / CONTROLADORES  ·  Node.js + Express       │
│  Routes: auth · users · assets · inventory               │
│          maintenance · disposal                          │
│  Middlewares: helmet · cors · express.json               │
├──────────────────────────────────────────────────────────┤
│  CAPA DE ACCESO A DATOS  ·  db.js                        │
│  PostgreSQL Pool (pg) — consultas parametrizadas         │
├──────────────────────────────────────────────────────────┤
│  CAPA DE DATOS  ·  PostgreSQL 15                         │
│  Tablas · ENUMs · Triggers · Transacciones               │
└──────────────────────────────────────────────────────────┘
```

¿Por qué esta arquitectura y no microservicios o MVC puro?

- Los módulos (auth, assets, inventory, etc.) son **rutas Express dentro del mismo proceso**, no servicios independientes. Microservicios requeriría múltiples procesos, bases de datos separadas y un orquestador (Kubernetes), complejidad innecesaria para el tamaño del proyecto.
- El backend no renderiza HTML; las "vistas" viven completamente en Flutter. Por tanto, el MVC clásico de servidor no aplica como patrón completo.
- La arquitectura por capas garantiza **separación de responsabilidades**, comunicación estrictamente vertical y facilidad de mantenimiento con un equipo pequeño.

2.2 Diseño de la Base de Datos

El modelo relacional centraliza la información en la tabla `assets` y extiende su trazabilidad y operación con tablas satélite:

```
users                     → control de acceso y sesiones
assets                    → catálogo maestro de activos fijos
asset_history             → log inmutable (audit trail) de cada activo
inventory_sessions        → cabeceras de toma física
inventory_session_baseline→ estado esperado al inicio del conteo (tabla puente)
inventory_verifications   → resultado real del conteo por activo
maintenance_requests      → solicitudes de mantenimiento preventivo/correctivo
disposal_requests         → solicitudes de baja con doble aprobación
```

Decisiones de diseño notables:

Decisión	Justificación
ENUMs en PostgreSQL (`asset_state`, `user_role`)	Impiden la inserción de valores inválidos a nivel de BD, sin necesidad de validación extra en el backend.
`ON DELETE CASCADE` en tablas satélite	Garantiza integridad referencial: al eliminar un activo, se purga automáticamente su historial, verificaciones y solicitudes.
Trigger `set_updated_at`	Actualiza `updated_at` automáticamente en cada UPDATE de `assets`, sin depender del código de la aplicación.
`pgcrypto` + `crypt()` para contraseñas	bcrypt nativo en la base de datos; la contraseña nunca viaja ni se almacena en texto plano.
`asset_code` sin FK en `inventory_verifications`	Permite registrar la verificación de un activo escaneado que aún no esté en el catálogo (activo "fantasma" detectado en campo).

Fragmento de `init.sql` — definición central:

```sql
-- Tipos enumerados que garantizan valores válidos a nivel de base de datos
CREATE TYPE asset_state AS ENUM (
  'activo', 'reubicado', 'noEncontrado',
  'obsoleto', 'enReparacion', 'paraBaja'
);
```

```sql
CREATE TABLE assets (
  code                        VARCHAR(50)   PRIMARY KEY,
  name                        VARCHAR(200)  NOT NULL,
  category                    VARCHAR(100)  NOT NULL,
  state                       asset_state   NOT NULL DEFAULT 'activo',
  created_at                  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
  updated_at                  TIMESTAMPTZ   NOT NULL DEFAULT NOW()
  -- ... más columnas
);

-- Trigger: el campo updated_at se mantiene automáticamente
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER assets_updated_at
  BEFORE UPDATE ON assets
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

2.3 Diseño de la Interfaz de Usuario

La aplicación sigue las guías de Material Design 3 con una paleta de color institucional verde (`#00804E`). La navegación principal es un `NavigationDrawer` lateral que adapta sus opciones al rol del usuario autenticado:

Sección	Roles con acceso
Dashboard	Todos
Activos	Administrador, Auxiliar
Usuarios	Administrador
Escanear	Auxiliar, Administrador
Notificaciones	Todos (filtradas por rol)
Reportes	Auditor, DAF, Administrador
Configuración	Administrador, Soporte TI

Pantallas principales:

- `LoginPage` — autenticación con indicador de intentos restantes y cuenta regresiva de bloqueo.
- `DashboardPage` — tarjetas de resumen: total activos, activos en riesgo, verificados este mes.
- `AssetsPage` — listado con búsqueda, filtros y acceso al detalle del activo con historial.
- `ScanPage` — cámara en vivo con `MobileScanner` para identificar activos por QR o código de barras.
- `ReportsPage` — generación de PDF/Excel con filtros avanzados.

2.4 Especificación Técnica y Seguridad

Aspecto	Implementación
Autenticación	Credenciales verificadas con `crypt()` de pgcrypto; sin JWT en v1 (sesión en memoria del cliente).
Bloqueo de cuenta	Tras N intentos fallidos (configurable), la cuenta se bloquea con `lock_until` en BD por X minutos.
Headers HTTP	`helmet` configura automáticamente X-Frame-Options, X-Content-Type-Options, CSP, HSTS, etc.
CORS	Habilitado globalmente; en producción debe restringirse al origen de la app.
Consultas SQL	100% parametrizadas (`$1, $2, ...`), sin concatenación de strings → previene SQL Injection.
Despliegue	Docker Compose con 2 contenedores (backend + BD) y volumen persistente `inventario_data`.

2.5 Infraestructura y Despliegue

```
# docker-compose.yml — estructura de contenedores
services:
  db:        # PostgreSQL 15-alpine, puerto 5432 interno
  backend:   # Node.js 18, expuesto en puerto 3000
             # depends_on con healthcheck (pg_isready)
```

El servicio `backend` solo arranca una vez que la base de datos pasa el `healthcheck`, evitando errores de conexión en el arranque en frío.

---

3. Etapa de Codificación

3.1 Paradigma y Stack Tecnológico

Componente	Tecnología	Versión	Justificación
Frontend/Móvil	Flutter + Dart	3.x / SDK ^3.8.1	Compilación nativa multiplataforma (Android, iOS, Web) desde una sola base de código.
Backend API	Node.js + Express	18 / 4.x	Ecosistema maduro, alta productividad para APIs REST, ideal para equipos pequeños.
Base de datos	PostgreSQL	15-alpine	RDBMS robusto con soporte nativo de ENUMs, arrays, pgcrypto y triggers.
Infraestructura	Docker + Compose	3.9	Entorno reproducible, fácil despliegue en cualquier servidor Linux.

3.2 Estructura del Código

```
proyecto-grado/
├── lib/
│   └── main.dart          ← Toda la lógica Flutter (modelos, estado, UI)
├── backend/
│   └── src/
│       ├── index.js       ← Punto de entrada Express
│       ├── db.js          ← Pool de conexiones PostgreSQL
│       └── routes/        ← Un archivo por dominio de negocio
│           ├── auth.js
│           ├── users.js
│           ├── assets.js
│           ├── inventory.js
│           ├── maintenance.js
│           └── disposal.js
├── docker/
│   └── init.sql           ← Schema + datos semilla
└── docker-compose.yml
```

3.3 Módulo de Estado — `AppState`

`AppState` extiende `ChangeNotifier` y es el núcleo reactivo de la aplicación Flutter. Centraliza datos y lógica de negocio del lado del cliente.

¿Por qué `ChangeNotifier` y no BLoC/Riverpod?
Para un proyecto de tamaño académico con un solo desarrollador, `ChangeNotifier` + `AnimatedBuilder` ofrece la misma reactividad con considerablemente menos boilerplate.

```dart
// AppState: núcleo reactivo de la app
class AppState extends ChangeNotifier {
  final List<AppUser>        users         = [];
  final List<Asset>          assets        = [];
  final List<AppNotification> notifications = [];

  AppUser? currentUser;       // Usuario autenticado actualmente
  AuthMode authMode = AuthMode.institutional; // institucional o local/demo

  // Cada vez que se llama notifyListeners(), persiste automáticamente en disco
  @override
  void notifyListeners() {
    super.notifyListeners();
    _save().catchError((e) => debugPrint('Persistence error: $e'));
  }
}
```

Por qué el override de `notifyListeners()`: garantiza que cada cambio de estado se persista en `SharedPreferences` sin requerir que cada método llame explícitamente a `_save()`.

3.4 Autenticación y Control de Acceso (RBAC)

El sistema implementa doble modo de autenticación: institucional (contra el backend real) y local (modo demo sin red).

```
// AppState — login con selección de modo
Future<String?> login(String username, String password) async {
  if (authMode == AuthMode.institutional) {
    return _loginWithBackend(username.trim(), password);
  }
  return _loginLocal(username.trim(), password);
}

// Modo institucional: el backend valida con bcrypt en PostgreSQL
Future<String?> _loginWithBackend(String username, String password) async {
  final response = await http.post(
    Uri.parse('$_backendUrl/api/auth/login'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'username': username, 'password': password}),
  ).timeout(const Duration(seconds: 10));

  // Respuestas codificadas: LOCK:segundos | WARN:restantes | INFO:mensaje
  final body = jsonDecode(response.body) as Map<String, dynamic>;
  if (response.statusCode == 200) {
    currentUser = AppUser.fromJson(body); // construye el modelo del usuario
    notifyListeners();
    return null; // null = éxito
  }
  // ...manejo de errores tipados
}
```

En el backend, la validación de credenciales usa `crypt()` de pgcrypto directamente en la consulta SQL, evitando que la contraseña salga de la base de datos:

```
// backend/src/routes/auth.js — validación con bcrypt en PostgreSQL
const { rows } = await pool.query(
  `SELECT id, username, full_name, email, roles, is_active, area,
          failed_attempts, lock_until,
          (password_hash = crypt($2, password_hash)) AS password_ok
   FROM users
   WHERE lower(username) = lower($1)`,
  [username.trim(), password],  // consulta parametrizada → sin SQL injection
);
```

¿Por qué `crypt($2, password_hash)` y no comparar hashes en Node?
La función `crypt` de pgcrypto extrae la sal del hash existente y re-hashea la contraseña enviada en la misma operación de consulta. Esto significa que:

1. La contraseña en texto plano nunca se almacena en ningún log del servidor.
2. No se necesita traer el hash a Node para compararlo; la BD devuelve solo un booleano (`password_ok`).

3.5 Gestión de Activos — Modelo y Trazabilidad

El modelo `Asset` incluye un campo `history` que acumula eventos inmutables de trazabilidad:

```
// Modelo de activo con historial embebido
class Asset {
  final String code;           // PK — código único del activo
  String name;
  AssetState state;            // activo | reubicado | noEncontrado | ...
  final List<AssetHistoryEvent> history; // log de cambios del activo

  // Constructor crea la lista vacía si no se provee
  Asset({ ..., List<AssetHistoryEvent>? history })
    : history = history ?? [];
}

// Cada cambio registra quién, cuándo y qué cambió
class AssetHistoryEvent {
  final DateTime timestamp;
  final String action;      // 'CREACION' | 'ACTUALIZACION' | 'VERIFICACION'
  final String detail;      // descripción legible del cambio
  final String performedBy; // username del ejecutor
}
```

El método `updateAsset` en `AppState` construye el mensaje del evento solo con los campos que realmente cambiaron:

```
void updateAsset(Asset asset, { required String performedBy,
    String? newResponsible, String? newLocation, AssetState? newState }) {
  final changes = <String>[];
  if (newResponsible != null && newResponsible != asset.responsible) {
    changes.add('Responsable: ${asset.responsible} -> $newResponsible');
    asset.responsible = newResponsible;
  }
  // ... otros campos
  if (changes.isNotEmpty) {
    asset.history.add(AssetHistoryEvent(
      timestamp: DateTime.now(), action: 'ACTUALIZACION',
      detail: changes.join(' | '), performedBy: performedBy,
    ));
    notifyListeners(); // persiste y notifica la UI
  }
}
```

¿Por qué registrar solo los campos que cambiaron? Mantiene el historial limpio y legible: un auditor puede ver exactamente qué fue modificado sin ruido de campos sin cambios.

3.6 API REST — Backend Express

`index.js` es el punto de ensamblaje: instala middlewares de seguridad y registra las rutas:

```
// backend/src/index.js — ensamblaje de la API
const app = express();
app.use(helmet());        // headers de seguridad HTTP automáticos
app.use(cors());          // habilita Cross-Origin para el cliente Flutter
app.use(express.json());  // parsea body JSON

app.use('/api/auth',        authRouter);
app.use('/api/assets',      assetsRouter);
app.use('/api/inventory',   inventoryRouter);
// ... más rutas

// Manejador global de errores: captura cualquier excepción no controlada
app.use((err, _req, res, _next) => {
  console.error(err);
  res.status(500).json({ error: 'Error interno del servidor' });
});
```

El `db.js` centraliza el pool de conexiones; ninguna ruta crea conexiones individuales:

```
// backend/src/db.js — pool único compartido por todas las rutas
const pool = new Pool({
  host:     process.env.DB_HOST     || 'localhost',
  database: process.env.DB_NAME     || 'inventario',
  user:     process.env.DB_USER     || 'postgres',
  password: process.env.DB_PASSWORD || 'postgres',
});
module.exports = pool;
```

¿Por qué un pool compartido? Abrir una conexión TCP a PostgreSQL por cada request HTTP es costoso (~100 ms). El pool mantiene conexiones abiertas y las reutiliza, reduciendo la latencia a menos de 5 ms en consultas simples.

3.7 Escaneo de Activos — `ScanPage`

La pantalla de escaneo usa `MobileScanner` para leer QR y códigos de barras en tiempo real. Al detectar un código:

1. Busca el activo en `AppState` (local o vía backend).
2. Si lo encuentra: navega al detalle del activo.
3. Si no lo encuentra: ofrece registrar el activo nuevo o reportar la anomalía.

```
// ScanPage — callback cuando se detecta un código
MobileScanner(
  onDetect: (capture) {
    final code = capture.barcodes.first.rawValue;
    if (code == null) return;
```

```
      final asset = state.findAsset(code);
      if (asset != null) {
        Navigator.push(context, MaterialPageRoute(
          builder: (_) => AssetDetailPage(asset: asset, state: state),
        ));
      } else {
        // Activo no encontrado → reporte de anomalía
        state.reportMissingAsset(scannedCode: code, notes: '');
      }
    },
  )
```

3.8 Sistema de Notificaciones con RBAC

Las notificaciones son visibles solo para los roles destino:

```
// AppState — filtro de visibilidad por rol
bool _notifVisibleToMe(AppNotification n) {
  if (currentUser == null) return false;
  if (n.fromUser == currentUser!.username) return true; // siempre veo las mías
  if (n.toRoles.isNotEmpty) {
    // solo si tengo alguno de los roles destinatarios
    return currentUser!.roles.any((r) => n.toRoles.contains(r.name));
  }
  return currentUser!.roles.contains(UserRole.administrador);
}
```

¿Por qué filtrar en cliente y no solo en servidor? La versión actual usa modo local (SharedPreferences) donde no hay servidor de notificaciones. El filtro en cliente replica la misma lógica que el backend aplicaría en la versión con BD.

---

4. Etapa de Pruebas e Implementación

4.1 Estrategia de Pruebas

Tipo	Herramienta	Alcance
Análisis estático	`flutter analyze`	Todo el código Dart — detecta warnings, hints y errores de tipo.
Pruebas de humo	Ejecución manual en emulador Android y navegador web	Flujos críticos: login, CRUD de activos, escaneo, reportes.
Pruebas de seguridad	Revisión manual de consultas SQL	Verificación de uso de parámetros (`$1, $2`) en todas las rutas.
Pruebas de integración	Postman / curl contra el backend dockerizado	Endpoints: `/api/auth/login`, `/api/assets`, `/api/inventory`.

4.2 Casos de Prueba Representativos

ID	Caso	Resultado Esperado	Estado
CP-01	Login con credenciales correctas	Token de sesión devuelto, redirección al dashboard	✅ Pasó
CP-02	Login con contraseña incorrecta (3 veces)	Cuenta bloqueada 15 min, mensaje de bloqueo con cuenta regresiva	✅ Pasó
CP-03	Registro de activo con código duplicado	Error 400 de la BD / validación de duplicidad en la UI	✅ Pasó
CP-04	Escaneo de QR de activo existente	Navegación al detalle del activo sin recargar la lista	✅ Pasó
CP-05	Generación de reporte PDF con 50 activos	Archivo PDF descargado con todos los activos filtrados	✅ Pasó
CP-06	Acceso a módulo de Usuarios con rol Auditor	Opción no visible en el menú (control por RBAC)	✅ Pasó
CP-07	Backend caído, modo local activado	App carga datos desde SharedPreferences sin errores	✅ Pasó

4.3 Resultado del Análisis Estático

```
$ flutter analyze
Analyzing proyecto-grado...
No issues found!  (ran in 4.2s)
```

4.4 Guía de Implementación

Prerrequisitos

- Docker Desktop ≥ 4.x instalado y en ejecución.
- Flutter SDK ≥ 3.8 (para compilar la app cliente).
- Android Studio / Xcode (para despliegue en dispositivo físico).

Paso 1 — Levantar el Backend y la Base de Datos

```
# Desde la raíz del proyecto
docker compose up -d

# Verificar que los contenedores estén saludables
docker compose ps
```

Esto inicia:

- `inventario_db`: PostgreSQL 15 con el schema inicializado desde `docker/init.sql`.
- `inventario_backend`: API REST accesible en `http://localhost:3000`.

Paso 2 — Verificar el Backend

```
curl http://localhost:3000/health
# Respuesta esperada: {"status":"ok"}
```

Paso 3 — Compilar y Ejecutar la App Flutter

```
# Instalar dependencias
flutter pub get

# Ejecutar en el dispositivo/emulador conectado
# El BACKEND_URL apunta al servidor AWS de producción por defecto
flutter run

# Para apuntar a un backend local (emulador Android):
flutter run --dart-define=BACKEND_URL=http://10.0.2.2:3000
```

Paso 4 — Compilar APK de Distribución

```
flutter build apk --release \
  --dart-define=BACKEND_URL=http://TU_IP_SERVIDOR:3000
```

Credenciales de Demo

Usuario	Contraseña	Rol
`admin`	`admin123`	Administrador + Soporte TI
`auxiliar`	`aux123`	Auxiliar de Inventario
`auditor`	`audit123`	Auditor
`daf`	`daf123`	Dirección Administrativa y Financiera
`resp`	`resp123`	Responsable de Área

4.5 Entorno de Producción

El backend está desplegado en una instancia AWS EC2 y es accesible en `http://18.223.120.46:3000`. La app Flutter apunta a este servidor por defecto (configurable con `--dart-define=BACKEND_URL`).

Para producción real se recomienda:

1. Habilitar HTTPS con un certificado TLS (Let's Encrypt o AWS Certificate Manager).
2. Colocar el backend detrás de un reverse proxy (Nginx).
3. Restringir el CORS al dominio de la app web.
4. Rotar las credenciales de base de datos y eliminar las cuentas demo.

Documento generado a partir del análisis del código fuente del repositorio.
Última actualización: Abril 2026